

Decorators in TypeScript



SoftUni Team
Technical Trainers



SoftUni



Software University

<http://softuni.bg>

Table of Contents

1. Introduction to Decorators
2. Syntax and Basic Usage
3. Types of Decorators
4. Advanced Usage





Introduction to Decorators

Definition

- Used in frameworks like **Angular**, **MobX** and others
- They are used to extend a functionality or add meta-data
- Use the form **@example** where **example** must evaluate to function that will be called at runtime



```
function example(target) {  
  // some code logic  
}
```

- We can decorate **five** different **things**:
 - Class definitions, properties, methods, accessors, parameters
- The function that we implement is **dependent** on the **thing** we are **decorating**
- The **arguments** required to **decorate a class** are different to the **arguments** required **to decorate a method**



Enable Decorators

- In the **tsconfig.json** file:

```
{  
  "compilerOptions": {  
    "experimentalDecorators": true,  
    "emitDecoratorMetadata": true  
  }  
}
```

```
tsc --experimentalDecorators --emitDecoratorMetadata
```

- There is well **defined order** to how decorators are applied:
 - **Parameter Decorators**, followed by **Method**, **Accessor**, or **Property Decorators** are applied for each instance member
 - **Parameter Decorators**, followed by **Method**, **Accessor**, or **Property Decorators** are applied for each static member
 - **Parameter Decorators** are applied for the **constructor**
 - **Class Decorators** are applied for the **class**



Syntax and Basic Usage

Syntax and Basic Usage

```
function classDecorator(constructor: Function) {  
    console.log("Class decorator called.");  
}
```

```
@classDecorator  
class ExampleClass {}
```

```
function methodDecorator() {  
    console.log("Method decorated.");  
}  
  
class ExampleClass {  
    @methodDecorator  
    exampleMethod(){}  
}
```

- Subclasses do not inherit the decorators of the super class
- Every subclass needs to be decorated on its own

```
function classDecorator(constructor: Function) {  
    console.log(`Called on ${constructor.name}`);  
}  
  
@classDecorator  
class ExampleClass {}  
// Called on ExampleClass  
  
class DerivedClass extends ExampleClass {}  
// No output, since decorators are not inherited
```



Types of Decorators

- **Class Decorators**
 - **Method Decorators**
 - **Accessor Decorators**
 - **Property Decorators**
 - **Parameter Decorators**
 - **Decorator Factories**
- 



Class Decorators

Definition

- **Class Decorator** is added just before the class declaration
- The Class Decorator receives the **constructor** of the class as a parameter
- Used to **observe, modify** or **replace** a class definition
- If the Class Decorator **returns a value**, it will **replace** the class declaration with the **provided constructor function**



Example: Class Decorator

```
function Frozen(constructor: Function) {  
    Object.freeze(constructor);  
    Object.freeze(constructor.prototype);  
}
```

@Frozen is a class decorator

```
@Frozen  
class Person {  
    constructor(private name: string) { }  
}
```

Example: Overwriting Class Definition

```
class Person { constructor(public name:string){ } }

@overwrite class Student extends Person {
  constructor(name: string, public age: number) { super(name); }
}

export function overwrite(constructor: Function) {
  return (function () { return { test: 20}} ) as any;
}

let student = new Student('George', 30);
console.log(student);
console.log(student instanceof Person);
console.log(student instanceof Student);
```

```
// {test: 20}
// false
// false
```


Example: Extending Class Definition

```
@addTitle class Person { constructor(public name:string){ } }

function addTitle<T extends new(...args: any[]) => {}>(constructor: T) {
  return class extends constructor {
    constructor(...args: any[]) {
      super('Sir/Madam ' + args[0], ...args.slice(1));
    }
  }
}

let person = new Person('George')
console.log(person.name)           // Sir/Madam George
```

The Person class constructor type

Example: Generic Class Decorators

```
@addTitleAbstract abstract class Test {}  
@addTitleGeneric class Person extends Test { constructor(public name: string) {super();}}  
@addTitleGeneric class Other { constructor(public lastName: string) { } }
```

Constructor type

```
function addTitleGeneric<T extends (new (...args: any[]) => {})>(constructor: T) {  
  return class extends constructor {  
    constructor(...args: any[]) { super('Sir/Madam ' + args[0], ...args.shift()); }  
  }  
}
```

Abstract constructor type

```
function addTitleAbstract<T extends (abstract new (...args: any[]) => {})>(constructor: T) {  
  abstract class Anonymous extends constructor { v = 20; }; return Anonymous;  
}
```

```
interface Test { v: number};  
let person = new Person('George');  
console.log(person.name);  
console.log(person.v)
```

Need to extend the Test class interface to let TS know about the new property

```
// Sir/Madam George  
// 20
```



Method Decorators

Definition

- 
- The method decorator function takes **three** arguments:
 - **target** - the parent **class**
 - **key** - the **name** of the function
 - **descriptor** - the **PropertyDescriptor** of the method
 - **descriptor.value** - the method itself

```
function disableEnumerable (  
    target: Object,  
    key: string,  
    descriptor: PropertyDescriptor) {  
    descriptor.enumerable = false;  
};
```

Example: Method Decorator

```
class Num {  
    constructor(private _number: number) { }  
    @add10 getNumber() { return this._number; }  
}  
  
function add10(target: Object, key: string, descriptor: PropertyDescriptor) {  
    const original = descriptor.value;  
    descriptor.value = function (...args: any[]) {  
        let result = original.apply(this, args);  
        return result += 10;;  
    };  
    return descriptor;  
};  
  
let num = new Num(20);  
console.log(num.getNumber()); // 30
```



Accessor Decorators

Definition

- TypeScript **does not allow** decorators on **both** the **getter** and the **setter**
- The **Property Descriptor** combines **both get** and **set** not each declaration separately
- Takes the following **three** arguments:
 - **target**
 - The **constructor function of the class** for a static members
 - The **prototype of the class** for instance members
 - **key** - The **name** of the member
 - **descriptor** - The **Property Descriptor** for the member



Example: Accessor Decorator

```
class Point {  
  constructor(private _x: number, private _y: number) { }  
  @double set x(value: number) { this._x = value; }  
  @double set y(value: number) { this._y = value; }  
}  
  
function double(target: any, key: string, descriptor: PropertyDescriptor) {  
  let originalSet = descriptor.set;  
  descriptor.set = function(val: any) {  
    originalSet?.call(this, val * 2);  
  }  
};  
  
let p = new Point(20, 20);  
p.x = 2; p.y = 3;  
console.log(p);  
  
// Point {_x: 4, _y: 6}
```


Example: Static Accessor Decorator

```
class Circle {  
  constructor(private _radius: number) { }  
  
  @format static get PI(){ return 3.1415 }  
}  
  
function format(target: any, key: string, descriptor: PropertyDescriptor) {  
  let original = descriptor.get;  
  descriptor.get = function() {  
    let result = original?.call(this);  
    return `PI is ${result.toFixed(2)}`;  
  }  
};  
  
let rect = new Circle(5);  
console.log(Circle.PI);  
  
// PI is 3.14
```



Property & Parameter Decorators

Definition

- 
- **Property decorator** can only be used to **observe** that a property with a specific name has been declared
 - **Cannot be used to modify the value of a property**
 - The **return value** of the decorator is also **ignored**
 - **Parameter decorators** can only be used to **observe** that a parameter has been declared on a method
 - **Cannot be used to modify the value of the parameter**
 - The **return value** of the decorator is also **ignored**

These two decorators are rare in practice. Know them for completeness, but they are not commonly used



Decorator Factories

Definition

- **Function** that **returns** a **decorator function**
- Gives the flexibility to pass **custom data** to the decorator function when needed



```
function enumerable(value: boolean) {  
  return function (  
    target: Object,  
    propertyKey: string,  
    descriptor: PropertyDescriptor) {  
      descriptor.enumerable = value;  
    };  
}
```

Decorator Factory

Decorator function

Example: Decorator Factory

```
class Name {
  constructor(private _name: string) { }
  @format('Hello, my name is %s')
  getName() { return this._name; }
}

function format(stringFormat: string) {
  return function (target: Object, key: string, descriptor: PropertyDescriptor) {
    const original = descriptor.value;
    descriptor.value = function (...args: any[]) {
      let value = original.call(this, ...args);
      return stringFormat.replace('%s', value);
    };
    return descriptor;
  }
}

let name = new Name('Peter');
console.log(name.getName());
```

// Hello, my name is Peter

- We can chain **multiple decorators** for each class declaration, method, property, accessor or parameter
- If multiple decorators / decorator factories exist:
 1. First all factories are called **in-order**:
log1 -> log2
 2. Then all the decorators are applied using **composition**:
log1(log2(constructor))

```
@log1()  
@log2()  
class Person { constructor(private _age:number = 5) { } }
```

Example: Multiple Decorator Factories

```
function log1() {  
  console.log("log1 factory evaluated");  
  return function (target: any, propertyKey: string, descriptor: PropertyDescriptor) {  
    console.log("log1 decorator executed");  
  };  
}  
  
function log2() {  
  console.log("log2 factory evaluated ");  
  return function (target: any, propertyKey: string, descriptor: PropertyDescriptor) {  
    console.log("log2 decorator executed ");  
  };  
}  
  
class ExampleClass {  
  @log1() @log2() method() { }  
}
```


- When decorators are **composed** (especially on getters or setters), the order of execution can become difficult to track
- Complex compositions are **rarely** used in practice
- Just remember:
 - In setters, decorators are applied in "**reverse order**"
 - In getters, the decorator composition order is **maintained**

Example: Complex Composition Setter

```
class Num {
  constructor(private _number: number) { }
  // (x => x - 2) -> (x => x * 2) -> (x => x + 5) -> original setter
  @modifyInput((x => x - 2)) @modifyInput((x => x * 2)) @modifyInput((x => x + 5))
  setNumber(val: number) { this._number = val; }
  getNumber() { return this._number; }
}

function modifyInput(mathFunc: (val: number) => number) {
  return function (target: Object, key: string, descriptor: PropertyDescriptor) {
    const original = descriptor.value;
    descriptor.value = function (...args: any[]) {
      return original.call(this, mathFunc(args[0]));
    };
  };
}

let a = new Num(5);
a.setNumber(5); console.log(a);
```

*// {_number: 11} => (((5)-2)*2)+5*

Example: Complex Composition Getter

```
class Num {  
  constructor(private _number: number) { }  
  setNumber(val: number) { this._number = val; }  
  // original getter -> (x => x + 5) -> (x => x * 2) -> (x => x - 2)  
  @modifyOutput((x => x - 2)) @modifyOutput((x => x * 2)) @modifyOutput((x => x + 5))  
  getNumber() { return this._number; }  
}  
  
function modifyOutput(mathFunc: (val: number) => number) {  
  return function (target: Object, key: string, descriptor: PropertyDescriptor) {  
    const original = descriptor.value;  
    descriptor.value = function (...args: any[]) {  
      let value = original.call(this, ...args);  
      return mathFunc(value);  
    };  
  };  
}  
  
let a = new Num(5); console.log(a.getNumber()); // 18 => (((5)+5)*2)-2
```



Advanced Usage

- Sometimes we might need some type information exposed at runtime for that we can use the **reflect-metadata** library

```
npm install reflect-metadata
```

- Adds an internal **[[Metadata]]** property to objects, that holds type information we can call at runtime
 - Requires "**emitDecoratorMetadata**": **true** in **tsconfig**
- We can use this runtime type information to implement more complex use cases like:
 - Dependency Injection
 - More robust validation

Example: Reflect-Metadata

```
import "reflect-metadata";

class Address { constructor(public city: string) { } }
class Person {
  constructor(private _address: Address) { }
  @validate set address(value: Address) { this._address = value; }
  get address() { return this._address; }
}

function validate<T>(target: any, propertyKey: string, descriptor: TypedPropertyDescriptor<T>) {
  let set = descriptor.set!;
  descriptor.set = function (value: T) {
    let type = Reflect.getMetadata("design:type", target, propertyKey);
    if (!(value instanceof type)) { throw new TypeError('Invalid type.')}
    set.call(this, value);
  };
}

let city = new Address('Tokyo');
let person = new Person(city);
console.log(person)
person.address = { city: 'New York'};
```

```
// {_address: Address {city: 'Tokyo'}}
// Runtime TypeError: Invalid type.
```

- Decorators are basically **functions**
- Add **additional functionalities** to a class or class members
- We can decorate **class declaration, methods, accessors, properties** and **parameters**
- To decorate different classes or class members the decorator functions takes **different arguments**



- Software University – High-Quality Education, Profession and Job for Software Developers
 - softuni.bg, about.softuni.bg
- Software University Foundation
 - softuni.foundation
- Software University @ Facebook
 - facebook.com/SoftwareUniversity



- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – <https://about.softuni.bg/>
- © Software University – <https://softuni.bg>

